MeshTac: Mesh-Aligned Pseudo-Labeling for Label-Efficient Contact Geometry and Pose Estimation

Anonymous submission

***Abstract*— Known object meshes can provide contact geometry labels for vision-based tactile sensors, but each real image must first be registered to its mesh. We propose MeshTac, which learns from a small registered set to generate pseudo-labels for additional real images. A teacher estimates the contact state, and a residual aligner corrects it using the tactile image and projected mesh depth. The aligner is trained on perturbed registered contacts without additional real labels. The aligned mesh supplies depth, normal and planar-pose targets for a student that predicts without mesh rendering. The same aligner can optionally refine pose at inference. We evaluate sensor-specific models on 12 ridge-pattern objects with two GelSlim-type sensors under matched training-label budgets. With 25 labeled contacts per object and no synthetic images, MeshTac reduces contact depth MAE by 21% and 36% relative to real-only training. On contact metrics, it matches real-only models trained with twice as many labels. Synthetic augmentation provides little further improvement within the training session but helps on a second session at the larger budget. A renderer fitted on all real training pairs reduces depth MSE by 24–26%, compared with 0–3% under matched budgets. Accurate mesh placement improves supervision on real contacts, while renderer comparisons must account for their real training labels.**

I. Introduction

Contact geometry and pose describe where an object touches a fingertip and how it is oriented. Vision-based tactile sensors such as GelSight, GelSlim and DIGIT [1]–[3] record these contacts as images of a deforming elastomer. Training a model to recover geometry from the images requires paired labels. A known object mesh can supply these labels, but each real contact must first be registered to the mesh.

Simulation and self-training offer two ways to use a small labeled set. Simulation generates labeled contacts from known geometry [4]–[6]. A renderer fitted on real image–geometry pairs can adapt their appearance to the target sensor, but those pairs consume part of the label budget. Self-training instead uses a predictor's outputs to label the remaining real images and can propagate its errors [7].

For known objects, we can use the unlabeled real images directly. Their appearance already matches the sensor; the unknown is the contact state—in-plane rotation, translation and indentation. A teacher trained on the registered set can estimate this state. However, even an exact mesh produces incorrect dense labels if its placement is wrong. Improving the state estimate is therefore necessary before using the mesh as supervision.

MeshTac learns this correction from the registered contacts (Fig. 1). We perturb their states and train a residual aligner to recover them from the real image and candidate mesh depth. Applied to the teacher's estimates, the aligner places the mesh before generating depth, normal and pose pseudo-labels. A student learns from these labels and predicts without mesh rendering.

Our contributions are:

- MeshTac, a method that converts known meshes into depth, normal and pose supervision for unlabeled real contacts through learned alignment, without generating tactile RGB images.
- A renderer study under matched supervision budgets and held-out-object conditions, distinguishing gains from simulated geometric coverage from those obtained with additional real training labels.
- An evaluation that links pseudo-label quality to student accuracy, tests label efficiency on two sensors and robustness on a second session, and reuses the aligner for test-time pose refinement.

II. Related Work

Tactile simulation and transfer. TACTO [4], Taxim [5], Tacchi [6] and physics-based rendering [8] model tactile images through optical response and contact geometry or deformation. Differentiable [9] and GPU-based [10] simulation support learning and control. Transfer methods translate tactile appearance [11], map real images to simulated depth [12], or generate tactile images from visual RGB-D [13]. Their data requirements differ. Our study concerns depth-to-RGB renderers fitted on real contact pairs and counts those pairs in the training-label budget.

Tactile representations. Pretrained encoders such as SITR, T3 and Sparsh [14]–[16] reduce the need to learn tactile features from scratch. MeshTac addresses the supervision for geometry and pose rather than the encoder: all predictors use the same frozen SITR backbone.

Pseudo-labeling and pose refinement. Self-training uses model predictions as labels [17], [18] and can reinforce teacher errors [7]. Known geometry offers an additional constraint. DeepIM [19] learns iterative pose corrections from observed and rendered images; Self6D [20] uses differentiable rendering to supervise pose on unlabeled RGB-D images. MeshTac learns corrections from tactile RGB and projected mesh depth, then uses the corrected state to supervise both dense geometry and pose on real images. Its main purpose is to improve training labels; mesh-assisted inference is optional.

III. Method: Mesh-Aligned Pseudo-Labeling

MeshTac uses a known mesh and object identity to label additional real contacts (Fig. 1). A teacher f_0, trained on K registered contacts per object, estimates their contact states.

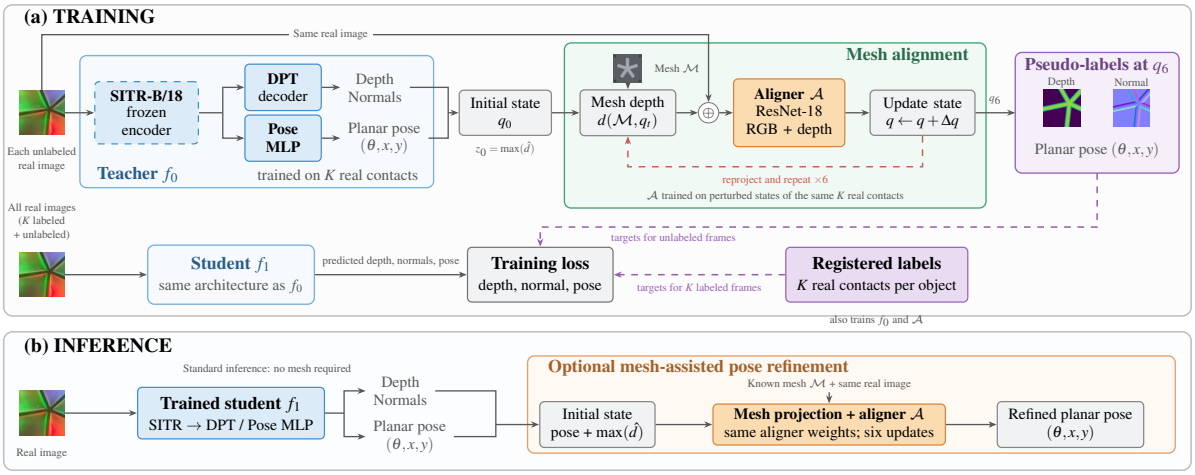


Fig. 1: **MeshTac training and inference.** (a) Teacher f_0 initializes mesh alignment to generate pseudo-labels for student f_1. Both use a frozen SITR encoder, DPT decoder and Pose MLP. (b) The student predicts without a mesh; optional refinement reuses aligner $\mathcal{A}$ with the real image and mesh. Thumbnails: a real $K = 25$ example.

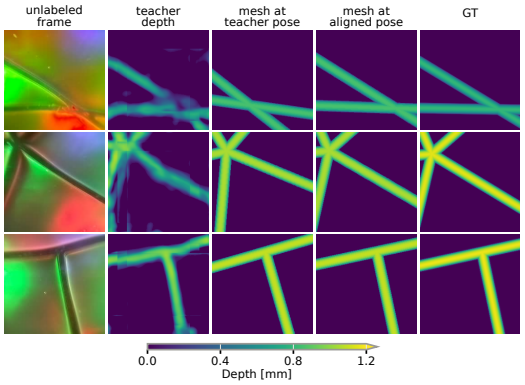


Fig. 2: **Mesh alignment for pseudo-label generation** ($K = 25$, sensor A). Columns: real image, teacher depth, mesh depth before and after alignment, and GT. Each example has the median aligned-label IoU for its object.

Aligner $\mathcal{A}$ corrects these estimates using the real images and projected mesh depth. The aligned mesh then supplies targets for student f_1.

Contact geometry. We represent a contact by $q = (\theta, x, y, z)$: in-plane rotation, translation and indentation. Let $d(\mathcal{M}, q)$ denote the depth projection of mesh $\mathcal{M}$ at this state. The projection gives depth and surface-normal labels; (θ, x, y) gives the planar-pose label. Alignment updates q while keeping the mesh fixed. Indentation determines the dense labels but is not a student pose output.

Learning the aligner. For a labeled image $\mathcal{I}$ with registered state q^*, we perturb the state by δ and project the mesh. The real image stays fixed. An ImageNet-initialized ResNet-18 receives the concatenated tactile RGB and candidate depth as four channels and predicts the correction:

$$\mathcal{A}\big(\mathcal{I}, d(\mathcal{M}, q^* + \delta)\big) \approx -\delta. \tag{1}$$

Sampling perturbations creates training pairs from the same K contacts without additional real labels or synthetic RGB images. We normalize each residual dimension and train with smooth-L1 loss. Sec. IV gives the perturbation ranges and training settings.

Generating pseudo-labels. The teacher's planar-pose prediction and maximum predicted depth initialize the state of an unlabeled image. We refine it over six iterations,

$$q_{t+1} = q_t + \mathcal{A}\big(\mathcal{I}, d(\mathcal{M}, q_t)\big), \tag{2}$$

with indentation clipped to $[0.3, 1.6]$ mm. The final state supplies the planar-pose target and determines the projected depth and normals (Fig. 2). We retain all pseudo-labeled training frames in the main configuration.

Student training and inference. Teacher and student have the same architecture but are trained separately. The student learns from real images with registered or mesh-derived labels. Its objective combines depth and normal MSE, cosine rotation loss and L1 translation loss, with learned weights within the dense and pose groups. The main method uses no synthetic RGB images; renderer augmentation is a separate variant. At inference, the student uses the tactile image and known object identity. If the mesh is available, the fixed aligner can also refine its pose (Sec. V-D).

IV. Experimental Setup

Sensors and geometric labels. Sensor A is a GelSlim-type fingertip with a 17.5 mm gel view and six colored LED emitters. A robot presses it against 12 ridge-pattern objects with known meshes. Ground-truth (GT) depth and normals are mesh projections at the recorded pose and describe registered object geometry rather than measured membrane deformation. Sensor B, GelSlim 4.0 [21], has a marker array, different illumination and a 12 mm view. We use its public dataset of the same 12 objects. We fit the meshes to its clipped, smoothed depth labels (0.018 mm mean residual) and use the resulting mesh projections as targets for all sensor-B methods.

Data splits and label budgets. Sensor A has 4,669 filtered frames; every tenth frame is reserved for validation (472 frames). A second session, recorded on another day after object re-placement, contains 854 reviewed test frames from 11 objects, with poses re-estimated from contact masks. Sensor B has 1,620 test frames. We select $K \in \{10, 25, 50, 100\}$

evenly spaced registered training contacts per object. The teacher, aligner and budget-matched renderer use this same set; the remaining training images are unlabeled. No validation or test images enter component training. The 472 sensor-A validation labels are additional to K and are used for checkpoint selection and the main budget comparison. Session 2 and sensor B provide separate test evaluations.

Predictor architecture and training. All predictors use a frozen SITR-B/18 encoder [14], a DPT decoder [22] for depth and normals, and a planar-pose MLP, with 224×224 inputs and 12M trainable parameters. A learned object embedding conditions only the pose head. Unless stated otherwise, experiments use sensor A. Main runs use 150 epochs; ablations, three-seed checks, sensor-B runs and the $K = 10$ learned renderer (full) use 50. The main teacher trains on the K real contacts only. In + *learned synth.*, teacher and student also train on synthetic images; the student's renderer is retrained on registered and aligned real pairs.

Aligner training. Perturbations span $\pm 8°$, ± 1.5 mm in-plane and ± 0.15 mm in indentation; half use 30% of these ranges. We train for 8,000 steps with batch size 64 and AdamW at learning rate 3×10^{-4}. The main aligner uses real images only. In + *learned synth.*, each batch contains 24 real and 40 synthetic examples from the learned renderer (K); the teacher and student also receive synthetic images.

Renderers and computation. The physically based renderer (PBR) uses Blender/Cycles, with illumination fitted to a no-contact image and contact parameters tuned manually. The learned renderer is a pix2pix GAN [23] mapping depth to RGB (Fig. 3). The budget-matched renderer trains on K real pairs per object. The full-set reference trains on all 4,197 real training pairs, exceeding K but excluding validation images; in the held-out-object study, it also sees real training pairs from the evaluation objects. Both renderers generate the same 5,345 simulated contacts. On an RTX 6000 Ada GPU, aligner training takes 20 min, labeling 0.5 s per frame and student training 1.3–2.2 h.

Baselines. *Real only* trains on K registered contacts. *PBR* and *learned renderer* add simulated contacts. *Self-training* uses teacher depth as the unlabeled targets, stitching predictions from five image windows; only registered contacts receive pose supervision. A variant also uses teacher pose targets. *L1 search* first fits the mesh to teacher depth, initialized at the teacher's pose. The learned renderer (K) generates RGB at 567 nearby poses; the lowest full-image L1 error against the real image selects the pose target. For each object, the lowest-error half of frames uses mesh depth and the rest teacher depth; all receive search pose targets.

Metrics and model selection. We report all-pixel depth MSE (mm^2), contact MAE within GT depth > 0 (mm), contact IoU (predicted depth > 0.05 mm versus GT depth > 0), normal angular error and rotation error (deg), and translation error (mm). Sensor-A budget and held-out-object comparisons report dense metrics at the checkpoint with lowest validation depth MSE and pose at the final epoch. Sensor-B and session-2 tests use the depth checkpoint; refinement selects the pose checkpoint by validation rotation error. Translation is mean absolute coordinate error for sensor-A budget and refinement results, and summed L1 over x, y for sensor-B and session-2 tests.

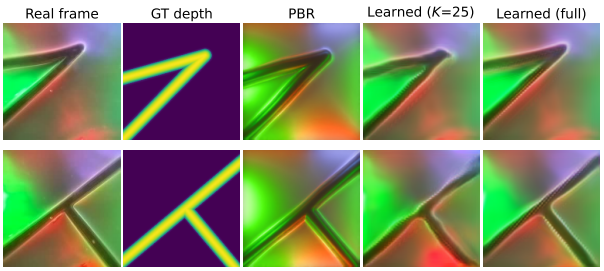


Fig. 3: **Tactile rendering at the recorded contact state.** Two real validation images, GT depth, and RGB images from PBR and learned renderers fitted on $K = 25$ contacts per object or the full real training set.

V. Experiments

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓	Trans. (mm)↓
10	Real only	0.0642	0.330	0.625	29.8	9.8	1.71
	Learned renderer (K)	0.0624	0.354	0.597	30.6	5.9	1.44
	Self-training	0.0574	0.341	0.592	28.3	6.6	1.67
	MeshTac (ours)	**0.0521**	0.293	0.660	**26.5**	4.7	**1.13**
	+ learned synth.	0.0559	**0.288**	**0.682**	27.2	**4.0**	1.14
	Learned renderer (full)	*0.0484*	*0.281*	*0.693*	*28.5*	*6.3*	*1.50*
25	Real only	0.0558	0.312	0.664	28.2	5.6	1.13
	Learned renderer (K)	0.0544	0.321	0.626	28.3	5.1	1.21
	Self-training	0.0508	0.307	0.628	26.5	6.3	1.51
	MeshTac (ours)	**0.0433**	**0.245**	0.726	**23.3**	**2.8**	**0.91**
	+ learned synth.	0.0450	0.249	**0.727**	24.0	3.7	1.00
	Learned renderer (full)	*0.0417*	*0.262*	*0.724*	*25.5*	*5.0*	*1.14*
50	Real only	0.0482	0.283	0.690	25.5	4.3	0.89
	Learned renderer (K)	0.0434	0.260	0.716	24.3	4.7	1.10
	Self-training	0.0381	0.255	0.706	22.7	5.5	1.30
	MeshTac (ours)	**0.0291**	**0.197**	0.761	**19.8**	**2.7**	**0.73**
	+ learned synth.	0.0307	0.206	**0.769**	20.4	3.7	0.95
	Learned renderer (full)	*0.0363*	*0.239*	*0.747*	*22.7*	*4.7*	*1.07*
100	Real only	0.0369	0.235	0.744	22.1	3.2	0.71
	Learned renderer (K)	0.0311	0.210	0.773	20.8	4.5	1.01
	Self-training	0.0285	0.211	0.750	19.7	4.9	1.16
	MeshTac (ours)	**0.0227**	**0.166**	**0.818**	**16.9**	**2.5**	**0.70**
	+ learned synth.	0.0232	0.168	0.803	17.1	3.7	0.90
	Learned renderer (full)	*0.0274*	*0.205*	*0.777*	*19.9*	*4.4*	*1.02*

TABLE I: **Label-budget comparison on sensor A** (472 validation frames). Matched methods use the same K registered contacts per object; self-training and MeshTac also use unlabeled training images. Italic: the full-set renderer reference, which exceeds K. Bold: best matched result, including the synthetic variant, at each K.

A. Label Efficiency

We vary K to compare mesh-aligned supervision with real-only training, renderer augmentation and self-training (Table I, Fig. 4). Matched methods use the same registered contacts; self-training and MeshTac also use the same unlabeled images.

MeshTac has lower contact MAE and normal error, and higher IoU, than the budget-matched baselines at every budget. At $K = 25$, it reduces depth MSE by 22% and contact MAE by 21% relative to real-only training, while matching or exceeding real-only contact accuracy at $K = 50$. The same comparison holds for $K = 50$ versus 100. MeshTac also lowers pose error on sensor A. Three-seed runs at

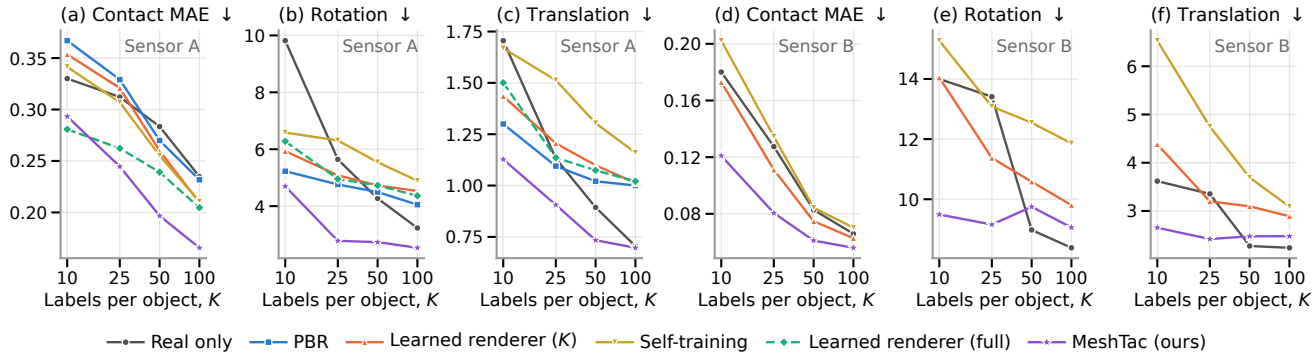


Fig. 4: **Label efficiency on two tactile sensors.** Contact MAE, rotation and translation error versus labeled contacts per object, K. Sensor A: (a–c), validation; sensor B: (d–f), test. Translation uses mean coordinate error in (c) and summed L1 in (f). Lower is better.

$K = 25$ give depth MSE 0.0433 ± 0.0004, compared with 0.0551 ± 0.0010 for real-only training; the improvement persists at the final epoch.

The baselines behave differently at small and large budgets. At $K \leq 25$, learned-renderer augmentation and self-training lower all-pixel MSE without consistently improving contact MAE or IoU; both improve contact metrics at larger K. PBR can improve rotation but does not improve contact reconstruction. These differences show why all-pixel MSE alone is insufficient to assess contact quality. Adding synthetic images to MeshTac gives small gains at $K = 10$ but no consistent further improvement across budgets.

Pseudo-label source		Pseudo-label accuracy				Student accuracy			
Depth source	Pose source	IoU ↑	MAE (mm)↓	Rot. (°)↓	Trans. (mm)↓	IoU ↑	MAE (mm)↓	Rot. (°)↓	Trans. (mm)↓
Teacher	–	0.56	0.32	–	–	0.62	0.31	7.5	1.6
Teacher	Teacher	0.56	0.32	4.9	1.0	0.62	0.31	6.0	1.4
Mesh	Teacher	0.47	0.40	6.0	1.1	0.50	0.38	5.7	1.2
Mesh	L1 search	0.63	0.30	2.5	0.80	0.65	0.29	5.5	1.3
Mesh	Aligner (MeshTac)	**0.72**	**0.23**	**1.1**	**0.67**	**0.72**	**0.25**	**4.0**	**1.1**

TABLE II: **Pseudo-label source ablation** ($K = 25$). Students share the architecture and training schedule. Left: pseudo-label accuracy against withheld training GT; right: student validation accuracy. L1 search uses mesh depth for the lowest-error half of frames and teacher depth otherwise. Shaded: MeshTac.

B. Ablation Study on Alignment Quality

We compare pseudo-label sources to separate the effects of mesh geometry, pose supervision and alignment (Table II). Students train for 50 epochs; results average three seeds except rows 2–3. Rows 1, 2 and 4 use a renderer-augmented teacher (4.9° pose error). Rows 3 and 5 share the real-only teacher (6.0°), directly testing the effect of alignment.

At the unrefined teacher pose (row 3), mesh labels have lower IoU than teacher depth (0.47 vs. 0.56), and their student performs worse than real-only training. Adding teacher pose supervision (row 2) mainly improves student rotation. L1 search (row 4) improves both label and student IoU, but the aligner (row 5) gives the best labels and student across the reported metrics despite its less accurate initial teacher. Relative to the unrefined state, alignment lowers label rotation error from 6.0° to 1.1° and translation error from 1.1 to 0.67 mm. Label improvements also hold on sensor B and across budgets (Fig. 5).

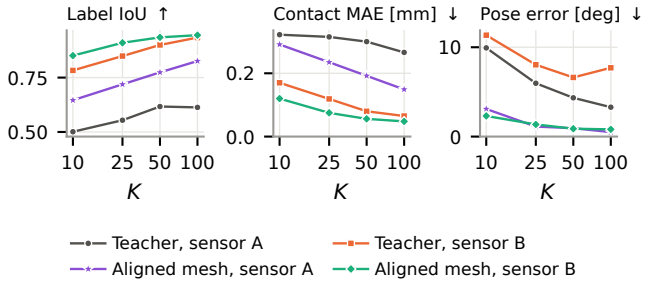


Fig. 5: **Pseudo-label quality on both sensors.** Teacher and aligned-mesh labels evaluated against withheld GT on unlabeled training images. GT is used only for evaluation. Curves show IoU, contact MAE and rotation error versus K.

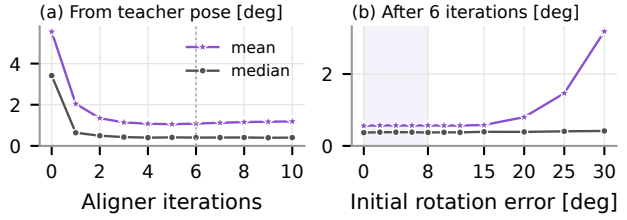


Fig. 6: **Aligner convergence** ($K = 25$, 472 validation frames). (a) Rotation error from teacher initialization versus update count; dotted line: six updates. (b) Rotation error after six updates from perturbed GT states, with random translation and indentation offsets. Shading: training rotation range (±8°).

The renderer's search score can favor an incorrect state: at $K = 25$, the search-selected pose has lower RGB L1 error than GT on 93% of frames. This limits image similarity as a criterion for geometric alignment. The aligner instead learns state corrections from known perturbations. From teacher initialization, mean rotation error reaches 1.1° after three updates (Fig. 6). Six updates also reduce 15° rotation perturbations to 0.6°, with simultaneous translation and indentation offsets, beyond the training rotation range. At 30°, the remaining error rises to 3.2°. Tested aligner settings and two random label selections preserve the gains; a second labeling round adds little. On unlabeled sensor-A training contacts, 1.6% of aligned poses exceed 5° error at $K = 25$,

mostly on one nearly symmetric object.

Sensor A: session-2 test (854 **frames), no retraining**						
	$K = 25$			$K = 100$		
Method	MAE↓	IoU↑	Trans.↓	MAE↓	IoU↑	Trans.↓
Real only	0.368	0.598	**3.94**	0.351	0.657	4.41
Self-training	0.335	0.598	4.60	0.300	0.707	3.91
MeshTac (ours)	**0.306**	0.654	4.41	0.288	0.707	4.06
+ learned synth.	0.310	**0.660**	4.21	**0.252**	**0.743**	**3.81**

Sensor B: test set (1,620 **frames), sensor-specific training**				
	$K = 10$	$K = 25$	$K = 50$	$K = 100$
Contact MAE, real only	0.180	0.127	0.083	0.066
Contact MAE, MeshTac (ours)	**0.121**	**0.081**	**0.061**	**0.056**
IoU, real only	0.780	0.847	0.899	0.932
IoU, MeshTac (ours)	**0.856**	**0.911**	**0.933**	**0.944**

TABLE III: **Second-session and second-sensor results.** Top: sensor-A models tested on session 2 without retraining. Bottom: sensor-specific training and testing on sensor B. Contact MAE and translation in mm; translation is summed L1 over x, y. Bold: best.

C. Robustness Across Sessions and Sensors

We test sensor-A models on another day's contacts after object re-placement, without retraining (Table III, top). Separately, we train each method on sensor B with unchanged aligner settings (bottom; Fig. 4d–f). The second evaluation tests whether the procedure remains useful with a different sensor design; it does not test zero-shot model transfer.

On session 2, MeshTac lowers contact MAE by 17–18% relative to real-only training and outperforms self-training. Adding synthetic frames provides no MAE improvement at $K = 25$, but reduces it to 0.252 mm at $K = 100$, a 28% reduction from real-only training. Translation remains less accurate across methods, indicating that better contact reconstruction does not by itself resolve the placement change.

On sensor B, MeshTac lowers contact MAE by 36% at $K = 25$ and matches real-only contact accuracy with twice as many labels. Reductions at other budgets range from 15 to 33%, exceeding renderer augmentation and self-training on contact MAE. The trend holds over three seeds at every budget and at $K = 25$ with the original public labels. Pose gains are less consistent at larger budgets (Fig. 4).

The examples in Fig. 7 show improved ridge continuity and junction placement. Together, the results indicate that aligned real contacts provide effective supervision within a session, while synthetic augmentation can add robustness to session changes when enough real pairs are available to fit the renderer.

D. Mesh-Assisted Test-Time Pose Refinement

When the mesh is available at inference, we reuse the labeling aligner to refine predictions from both real-only and MeshTac models. We apply six updates without retraining on session-1 validation and session-2 test images (Table IV). At $K \geq 25$, both predictors reach 0.4–1.2° mean rotation error within session 1, with translation error reduced by 20–50%. On session 2, mean rotation error is approximately 5–7° and median error 1.5–1.8°. Thus, the same aligner improves both predictors, but some contacts retain large errors under the session change. The mean–median gap motivates confidence estimates for refinement.

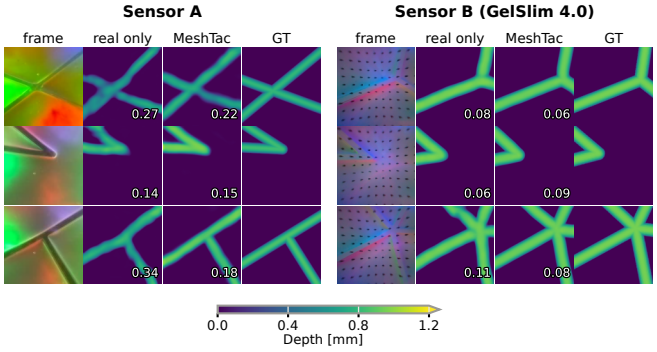


Fig. 7: **Contact-depth reconstruction at $K = 25$.** Real-only and MeshTac predictions with GT for three objects per sensor, using each object's median MeshTac contact-MAE frame. Depth and contact MAE are in mm.

	Validation (session 1)				Session 2			
	Real only		MeshTac		Real only		MeshTac	
K	pred.	+align	pred.	+align	pred.	+align	pred.	+align
Rotation error (deg)								
10	9.6	2.8	4.7	2.4	20.2	11.4	14.2	7.0
25	5.6	1.2	2.8	0.9	16.2	5.5	10.9	5.2
50	4.3	1.2	2.7	1.0	13.8	4.8	11.3	7.4
100	3.2	0.4	2.5	0.4	12.9	4.9	10.1	4.9
Translation error (mm)								
10	1.69	1.01	1.13	0.89	2.25	1.57	1.97	1.55
25	1.21	0.80	0.91	0.71	2.03	1.27	1.71	1.25
50	0.91	0.52	0.74	0.47	2.02	1.27	1.73	1.29
100	0.72	0.34	0.70	0.36	1.84	1.25	1.76	1.25

TABLE IV: **Pose error before and after test-time refinement.** *pred.*: before refinement; *+align*: six updates of the fixed aligner with the known mesh. Rotation is mean angular error; translation is mean coordinate error. No retraining.

Method	Depth MSE↓	Contact MAE↓	IoU ↑	Rot. (deg)↓
Real only (9 objects)	0.0665	0.356	0.582	64.8
PBR	0.0657	0.346	0.592	6.9
Learned renderer	0.0631	0.331	0.618	5.6
learned renderer, no held-out meshes	0.0647	0.340	0.604	50.5
learned renderer + background	**0.0581**	**0.323**	**0.633**	**4.8**
Learned renderer (full)	*0.0467*	*0.277*	*0.676*	*4.5*

TABLE V: **Renderer baselines on three held-out objects.** Matched methods exclude these objects' real contacts from training. The italic full-set reference uses their real training pairs. Simulated contacts use the held-out meshes unless stated otherwise. Bold: best matched result.

E. Renderer Baselines: Unseen Objects and Label Budgets

We evaluate renderer augmentation separately from MeshTac, first on objects excluded from real training, then under different renderer supervision budgets. For the held-out-object test, we remove three objects' real contacts while retaining their meshes for simulation (Table V). A control also removes their meshes; a background-conditioned renderer additionally receives a no-contact image from the session.

Simulated contacts reduce held-out-object rotation error from 64.8° to 5–7°. Excluding their meshes raises it to 50.5°,

while matched depth-MSE improvements remain at most 13%. The mesh control supports geometric coverage as the main source of the pose improvement; the smaller dense gains show that this does not fully address contact reconstruction.

For $K = 10$–25, the full-set renderer reduces depth MSE by 24–26% relative to real-only training, compared with 0–3% for the budget-matched renderer (Table I). The full-set reference uses additional real training pairs, not validation images. Its larger gain therefore does not establish a benefit from simulation under the same label budget. Renderer training labels must be included when comparing synthetic augmentation with other uses of real supervision.

VI. Discussion and Conclusion

MeshTac reduces the need to register real tactile contacts by learning mesh alignment from a small labeled set. The aligned mesh supplies depth, normal and pose targets for additional real images, and the student predicts without mesh rendering. Experiments on two sensors and a second session show improvements in contact reconstruction; the same aligner also improves pose at inference when the mesh is available.

The ablation indicates that contact-state accuracy is a bottleneck for mesh-derived supervision. An unrefined mesh gives worse labels than teacher depth, while correcting its placement improves both labels and student predictions. Unlabeled real images already contain the sensor's appearance, so alignment can improve their supervision without synthesizing RGB. Contact MAE and IoU are essential to this assessment: lower all-pixel MSE alone can conceal poor contact reconstruction.

Simulation offers a different benefit. The held-out-mesh control shows its value for pose coverage, and the second-session results show an additional benefit at the larger label budget. For known objects with unlabeled real contacts, these findings favor starting with mesh alignment and evaluating synthetic augmentation when coverage or session robustness is limiting. The renderer's real training pairs must be counted in that comparison.

The current evidence is limited to quasi-static contacts with 12 ridge-pattern objects, known identity and available meshes. Targets describe registered object geometry rather than membrane deformation. The reported K also excludes validation labels needed for checkpoint selection. Repeated ridges can make pose ambiguous despite accurate local geometry. Future work will investigate confidence estimates and symmetry-aware refinement, extend evaluation to dynamic contact and closed-loop manipulation, and study joint shape and state estimation when meshes are unavailable.

References

[1] W. Yuan, S. Dong, and E. H. Adelson, "Gelsight: High-resolution robot tactile sensors for estimating geometry and force," *Sensors*, vol. 17, no. 12, 2017.

[2] E. Donlon, S. Dong, M. Liu, J. Li, E. Adelson, and A. Rodriguez, "Gelslim: A high-resolution, compact, robust, and calibrated tactile-sensing finger," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018.

[3] M. Lambeta, P.-W. Chou, S. Tian, B. Yang, B. Maloon, V. R. Most, D. Stroud, R. Santos, A. Byagowi, G. Kammerer, D. Jayaraman, and R. Calandra, "Digit: A novel design for a low-cost compact high-resolution tactile sensor with application to in-hand manipulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 3838–3845, 2020.

[4] S. Wang, M. Lambeta, P.-W. Chou, and R. Calandra, "Tacto: A fast, flexible, and open-source simulator for high-resolution vision-based tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3930–3937, 2022.

[5] Z. Si and W. Yuan, "Taxim: An example-based simulation model for gelsight tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2361–2368, 2022.

[6] Z. Chen, S. Zhang, S. Luo, F. Sun, and B. Fang, "Tacchi: A pluggable and low computational cost elastomer deformation simulator for optical tactile sensors," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1239–1246, 2023.

[7] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," in *IJCNN*, 2020.

[8] A. Agarwal, T. Man, and W. Yuan, "Simulation of vision-based tactile sensors using physics based rendering," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1–7.

[9] J. Xu, S. Kim, T. Chen, A. R. Garcia, P. Agrawal, W. Matusik, and S. Sueda, "Efficient tactile simulation with differentiability for robotic manipulation," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1488–1498. [Online]. Available: https://proceedings.mlr.press/v205/xu23b.html

[10] I. Akinola, J. Xu, J. Carius, D. Fox, and Y. Narang, "Tacsl: A library for visuotactile sensor simulation and learning," *IEEE Transactions on Robotics*, 2025.

[11] W. Chen, Y. Xu, Z. Chen, P. Zeng, R. Dang, R. Chen, and J. Xu, "Bidirectional sim-to-real transfer for gelsight tactile sensors with cyclegan," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6187–6194, 2022.

[12] A. Church, J. Lloyd, R. Hadsell, and N. F. Lepora, "Tactile sim-to-real policy transfer via real-to-sim image translation," in *Proc. Conf. Robot Learning (CoRL)*, 2022, pp. 1645–1654.

[13] S. Zhong, A. Albini, P. Maiolino, and I. Posner, "Tactgen: Tactile sensory data generation via zero-shot sim-to-real transfer," *IEEE Transactions on Robotics*, vol. 41, pp. 1316–1328, 2025.

[14] H. Gupta, Y. Mo, S. Jin, and W. Yuan, "Sensor-invariant tactile representation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[15] J. Zhao, Y. Ma, L. Wang, and E. Adelson, "Transferable tactile transformers for representation learning across diverse sensors and tasks," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[16] C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam, "Sparsh: Self-supervised touch representations for vision-based tactile sensing," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[17] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[18] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves imagenet classification," in *CVPR*, 2020.

[19] Y. Li, G. Wang, X. Ji, Y. Xiang, and D. Fox, "DeepIM: Deep iterative matching for 6D pose estimation," in *ECCV*, 2018.

[20] G. Wang, F. Manhardt, J. Shao, X. Ji, N. Navab, and F. Tombari, "Self6D: Self-supervised monocular 6D object pose estimation," in *ECCV*, 2020.

[21] A. Sipos, W. van den Bogert, and N. Fazeli, "GelSlim 4.0: Focusing on touch and reproducibility," *arXiv preprint arXiv:2409.19770*, 2024.

[22] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 12 179–12 188.

[23] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.